PyTorch · Google Colab 기반 학습 과정 보고서

인공지능학과 2022480010 엄성호 | 2026 년 4 월 6 일

1. 과제 개요

이미지 3 장을 대상으로 다음 두 가지 조건을 동시에 만족하는 시스템을 구현하는 것을 목표로 하였다.

- **객체 분류 및 확률 출력:** 이미지 내 객체를 인식하고 클래스별 확률을 수치로 출력 (예: starfish 80.7%)
- **픽셀 단위 분류 시각화:** 각 픽셀이 어떤 객체에 속하는지 색상으로 구분하여 시각화

사용 환경은 Google Colab (GPU 런타임), 프레임워크는 PyTorch 및 HuggingFace Transformers 이며, 대상 이미지는 다음과 같다.

- 기찻길.jpg — 선로(railroad), 가드레일(guardrail), 산, 나무, 하늘
- 불가사리.jpg — 투명한 얕은 바다, 불가사리, 모래 바닥
- 다람쥐.jpg — 눈(falling snow) 오는 날 나무에 매달린 다람쥐

2. 사고 흐름 및 문제 해결 과정

과제를 수행하면서 단일 모델로는 두 조건을 동시에 만족하기 어렵다는 것을 확인하고, 각 모델의 한계를 보완하는 방향으로 반복적으로 개선하였다.

STEP 1. YOLOv8 시도 → 실패

Ultralytics GitHub(https://github.com/ultralytics/ultralytics)의 Quick Start 예제를 참고하여 YOLOv8 을 적용하였다. 확률값(confidence score)과 바운딩 박스를 동시에 출력해주는 구조가 과제 조건에 적합하다고 판단하였다.

```
Image 1/1 기찻길.jpg: 640x384 (no detections), 81.4ms
```

GitHub README 의 클래스 목록을 확인한 결과, YOLOv8 은 COCO 80 개 클래스 기반으로 학습되어 있으며 선로(railroad track), 불가사리(starfish) 등은 포함되지 않아 탐지에 실패하였다.

STEP 2. SegFormer 적용 → 부분 성공

HuggingFace 모델 허브(https://huggingface.co/nvidia/segformer-b2-finetuned-ade-512-512)의 "How to use" 예제 코드를 참고하여 SegFormer 를 적용하였다. 모델 페이지에서 ADE20K 150 개 클래스 목록을 확인하였고, "railroad track"이 포함되어 있어 기찻길 이미지에 적용하였다.

```
=== 감지된 객체 ===
rock: 25.9%
```

```
sky: 16.1%
mountain: 14.4%
dirt track: 4.7%
```

기찻길 이미지에서는 픽셀 단위 분류가 동작하였으나, 불가사리 및 다람쥐는 클래스 목록에 없어 적용이 불가능하였다.

STEP 3. CLIP 적용 → 유연한 클래스 지정

HuggingFace 모델 허브(https://huggingface.co/openai/clip-vit-base-patch32)의 예제 코드를 참고하였다. CLIP 은 클래스가 고정되어 있지 않고 사용자가 텍스트로 직접 입력할 수 있어 불가사리, 다람쥐 같은 커스텀 클래스에도 적용 가능하였다.

```
labels = ["starfish", "rock", "ocean", "sand", "coral"]
→ starfish: 99.3%
```

불가사리 이미지에서 99.3%의 높은 확률로 분류에 성공하였다. 그러나 CLIP 은 이미지 전체를 하나의 클래스로만 분류하므로 픽셀 단위 분류 조건을 만족하지 못하였다.

STEP 4. MobileSAM + CLIP 조합 발상

MobileSAM GitHub(https://github.com/ChaoningZhang/MobileSAM)의 README 예제를 참고하여 SAM 으로 영역을 먼저 분리한 뒤, 각 영역을 CLIP 으로 분류하는 발상을 해보았다. 그러던중 Grounded SAM 발견

STEP 5. Grounded SAM 도입 → 최종 파이프라인

Grounding DINO GitHub(https://github.com/IDEA-Research/GroundingDINO)와 HuggingFace Transformers 문서(https://huggingface.co/docs/transformers/model_doc/grounding-dino, https://huggingface.co/docs/transformers/model_doc/sam)를 참고하여 최종 파이프라인을 구성하였다.

```
기존 (MobileSAM + CLIP)
이미지 → SAM 이 무작위로 분리 → CLIP 이 사후 분류

Grounded SAM
이미지 + 텍스트 → Grounding DINO 가 위치 탐지 → SAM 이 해당 위치만 분리
```

GitHub Issue 에서 Grounding DINO 프롬프트는 클래스마다 마침표(.)로 구분해야 토크나이저가 정상 동작한다는 것을 확인하였다. 또한 SAM 의 input_boxes 차원을 [[x1,y1,x2,y2], ...] 형태로 맞춰야 에러가 나지 않는다는 것을 확인하였다.

철도 도메인 어휘(ballast, guardrail)와 기상 표현(falling snow) 등을 직접 조사하여 프롬프트 품질을 높였다.

- 기찻길: "railroad. guardrail. tree. mountain. ballast. sky."
- 불가사리: "the orange starfish. Sand under the sea."
- 다람쥐: "the squirrel. The tree trunk. snow."

3. 참고 자료

GitHub

- Ultralytics YOLOv8: https://github.com/ultralytics/ultralytics
- Grounding DINO: https://github.com/IDEA-Research/GroundingDINO
- Segment Anything (SAM): https://github.com/facebookresearch/segment-anything
- MobileSAM: https://github.com/ChaoningZhang/MobileSAM
- Grounded SAM: https://github.com/IDEA-Research/Grounded-Segment-Anything

HuggingFace 모델 허브 및 문서

- SegFormer: https://huggingface.co/nvidia/segformer-b2-finetuned-ade-512-512
- CLIP: https://huggingface.co/openai/clip-vit-base-patch32
- Grounding DINO 문서: https://huggingface.co/docs/transformers/model_doc/grounding-dino
- SAM 문서: https://huggingface.co/docs/transformers/model_doc/sam
- 시각화:https://github.com/huggingface/notebooks/blob/main/examples/segment_anything.ipynb